UNIVERSIDAD SERGIO ARBOLEDA

LENGUAJES DE PROGRAMACION

EJERCICIOS CAPITULO 2 FLEX Y BISON

DOCENTE

JOAQUIN FERNANDO SANCHEZ CIFUENTES

JULIAN CAMILO CARO HERRERA

SOLUCION EJERCICIOS

PRIMER PUNTO

Para este ejercicio se quiere hacer que salgan numeradas las líneas del archivo de entrada, que permita incluir más archivos y que se mantenga el control correcto del flujo de entrada. A continuación, se verá el paso a paso para realizar dicho objetivo.

Primer paso:
A continuación, en la figura 1 se ve la creación del archivo ejercicio1.l, donde se encuentra exactamente el mismo código proporcionado por el libro para hacer el ejemplo 2-3. Por lo tanto, acá solo se verá la modificación para poder lograr que funcione correctamente.

CREACION DEL ARCHIVO

```
julian@julian-VirtualBox:~/ejerciciosCAP2$ ls
ejercicio1.l
```

Figura 1 (Creación del archivo)

Segundo paso:

Con base al código proporcionado se desarrolló la siguiente expresión regular para que pueda capturar líneas completas desde el inicio hasta el salto de línea.

MODIFICACION AL CODIGO

```
^[^\n]+\n {
    fprintf(yyout, "%4d %s", lineno++, yytext);
}
```

Figura 2(Modificación del código para que cumpla los requerimientos)

En la figura anterior se puede ver que el operador ^ lo que hace es restringir la coincidencia al inicio de línea, mientras que [^\n]+ captura el contenido textual sin contar el salto, y" \n " garantiza que se trate de una línea completa. Luego se imprime el número de línea formateado a cuatro espacios y luego incrementa el contador, utilizando la variable automática yytext proporcionada por Flex.

ENTRADAS.TXT

En las siguientes dos figuras (3 y 4) se observan las 2 entradas de texto para comprobar que si funciono correctamente la implementación anterior.

```
  GNU nano 7.2                                entrada.txt *
Hola mundo
#include "otro.txt"
Ultima linea
```

Figura 3 (Entrada.txt)

OTRO.TXT

```
  GNU nano 7.2                                  otro.txt *
Linea A desde otro archivo
Linea B desde otro archivo
```

Figura 4 (Otro.txt)

El lexer procesa líneas completas numeradas. Cuando encuentra un #include "archivo" cambia el buffer de lectura al archivo incluido.

COMPILACION Y EJECUCION

Finalmente, después de haber creado todo correctamente compila y ejecuta como se ha visto en anteriores clases

```
julian@julian-VirtualBox:~/ejerciciosCAP2$ nano ejercicio1.l
julian@julian-VirtualBox:~/ejerciciosCAP2$ flex ejercicio1.l
gcc lex.yy.c -o ejercicio1 -lfl
./ejercicio1 entrada.txt
```

Figura 5(Compilación y ejecución)

RESULTADO

Finalmente se obtuvo el siguiente resultado:

```
1 Hola mundo
1 Linea A desde otro archivo
2 Linea B desde otro archivo
3 Ultima linea
```

Figura 6 (Resultado)

Al procesar el archivo principal, el lexer numera cada línea. Cuando encuentra un #include, cambia temporalmente al archivo incluido usando un buffer separado y reinicia el contador de líneas (lineno). Cada línea del archivo incluido se imprime con numeración local. Al terminar, el lexer retorna al archivo original y restaura el número de línea previo, haciendo que la numeración del archivo principal continúe correctamente.

SEGUNDO PUNTO

CODIGO

```
static unsigned symhash(char *sym) {
    unsigned int hash = 0;
    unsigned c;
    while ((c = *sym++)) hash = hash*9 ^ tolower(c);
    return hash;
}
%}
```

Se modificó la función de dispersión symhash() para normalizar cada carácter mediante tolower(), asegurando que palabras que difieran solo en capitalización generen el mismo valor hash.
Además, se reemplazó strcmp() por strcasecmp() en lookup(), permitiendo comparaciones insensibles a mayúsculas/minúsculas dentro de la tabla de símbolos.
Estas dos modificaciones garantizan unicidad léxica sin duplicar memoria ni alterar la estructura de la tabla hash.

ENTRADA1.TXT

```
  GNU nano 7.2                                  entrada1.txt
Hola Mundo
Esto es un Test
hola mundo
```

ENTRADA2.TXT

```
  GNU nano 7.2                                  entrada2.txt
El perro corre
el Perro duerme
```

Como en el anterior texto se toman 2 archivos de texto, para que al ejecutar la implementación se tengan en cuenta. Y que de tal forma tome ambos.

Compilación

```
julian@julian-VirtualBox:~/ejerciciosCAP2$ ./ejercicio2 entrada1.txt entrada2.txt
```

RESULTADO

```
Mundo entrada1.txt:1
El entrada2.txt:1
es entrada1.txt:1
Test entrada1.txt:1
un entrada1.txt:1
Esto entrada1.txt:1
duerme entrada2.txt:1
corre entrada2.txt:1
Hola entrada1.txt:1
perro entrada2.txt:1
```

Las modificaciones realizadas permiten que la tabla de símbolos tome las palabras de mayúsculas y minúsculas, tanto en el cálculo del hash como en la comparación de cadenas, cumpliendo con los requerimientos del ejercicio.

TERCER PUNTO

Para este tercer punto se utilizó la técnica de encadenamiento donde cada posición de la tabla hash almacena un puntero a una lista enlazada de símbolos.Cuando ocurre una colisión, el nuevo símbolo se inserta en la lista correspondiente al bucket calculado por la función hash. Esto con la información proporcionada en el libro. También con base al código proporcionado

```
struct symbol *symtab[NHASH];
char *curfilename;
```

Ahora cada bucket es un puntero. Lo que Permite múltiples entradas por posición. y El tamaño lógico ya no está limitado por NHASH.

```
struct symbol {
    char *name;
    struct ref *reflist;
    struct symbol *next;   /* chaining */
};
```

También se añadió un puntero de encadenamiento que enlaza símbolos del mismo índice,por lo que forma una lista enlazada

```
for (sp = symtab[h]; sp != NULL; sp = sp->next) {
    if (strcasecmp(sp->name, sym) == 0)
        return sp;
}
```

```
sp->next = symtab[h];
symtab[h] = sp;
```

Con las 2 figuras anteriores lo que se modifico fue que Se calcule el índice h = symhash(sym) % NHASH también que recorra la lista del bucket y Si no se encuentra el símbolo, se reserva memoria con mallocSe inserta al inicio de la lista

Compilación

```
julian@julian-VirtualBox:~/ejerciciosCAP2$ flex ejercicio3.l
gcc lex.yy.c -o ejercicio3 -lfl
```

Ejecución

```
julian@julian-VirtualBox:~/ejerciciosCAP2$ ./ejercicio3 entrada3.txt entrada4.txt
```

Resultado

```
mundo entrada4.txt:1 entrada3.txt:1
gato entrada4.txt:1 entrada3.txt:1
Hola entrada3.txt:1
final entrada4.txt:1
Perro entrada4.txt:1 entrada3.txt:1
```

El programa ya no presenta overflow.Se almacenan múltiples referencias Las colisiones se resuelven mediante listas enlazadas. No hay límite práctico fijo de inserción.